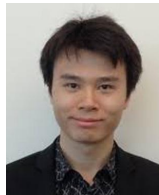


# Performance and Revenue Analysis of Hybrid Cloud Federations with QoS Requirements



## ***Bowen Song***



## Marco Paolieri

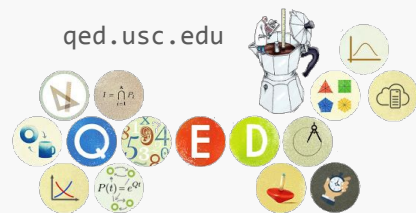


**Leana Golubchik**

(Quantitative Evaluation + Design Group)

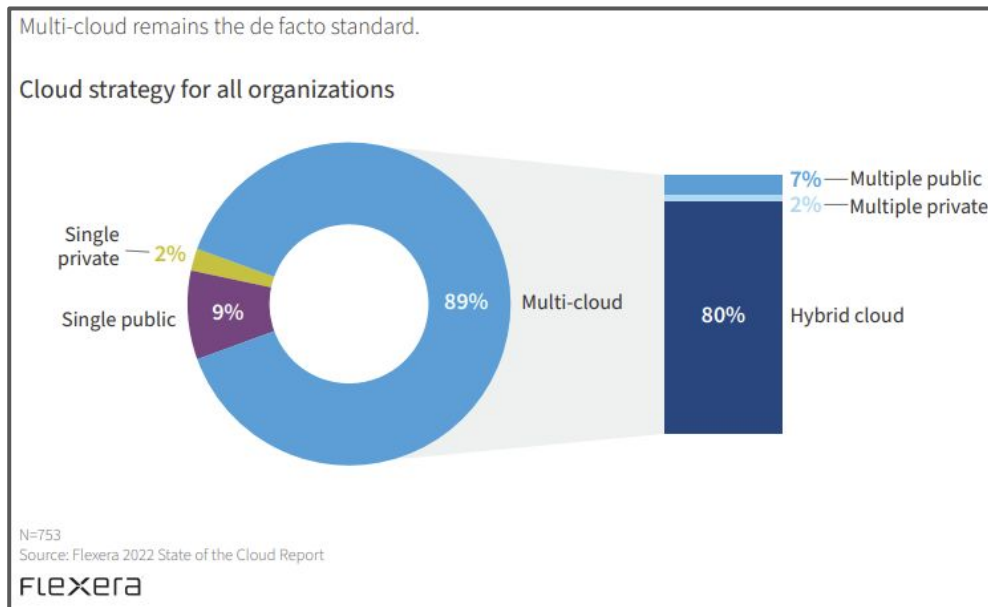
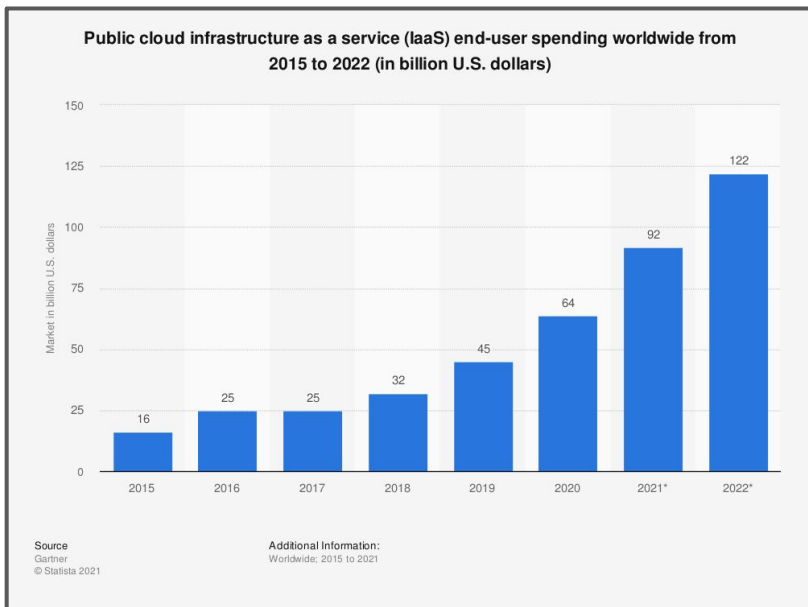


IEEE Cloud, July 11th, 2022



# Market of Infrastructure-as-a-Service

- Scalable computing, storage, and networking resources on demand.

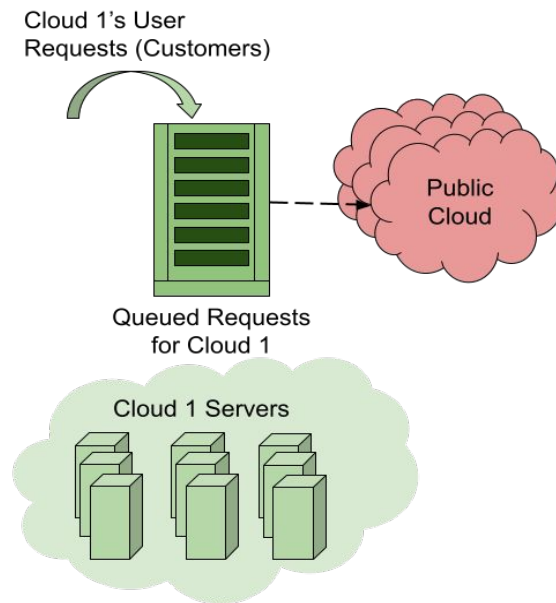


# Hybrid Cloud

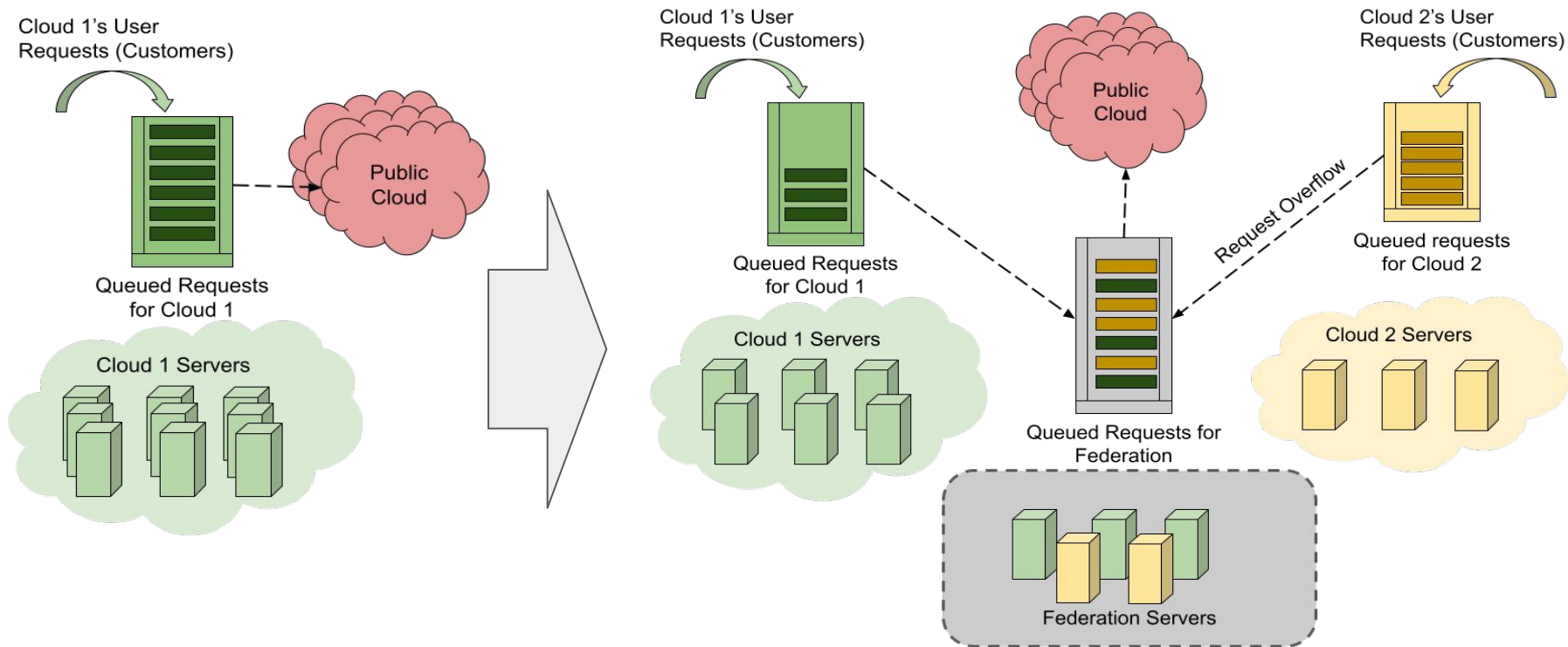
- An **under-provisioned** cloud can forward requests to public clouds
- Address **variable workloads** and **peak loads**

Does not help

- Over-provisioned clouds
- Long-term workload increase



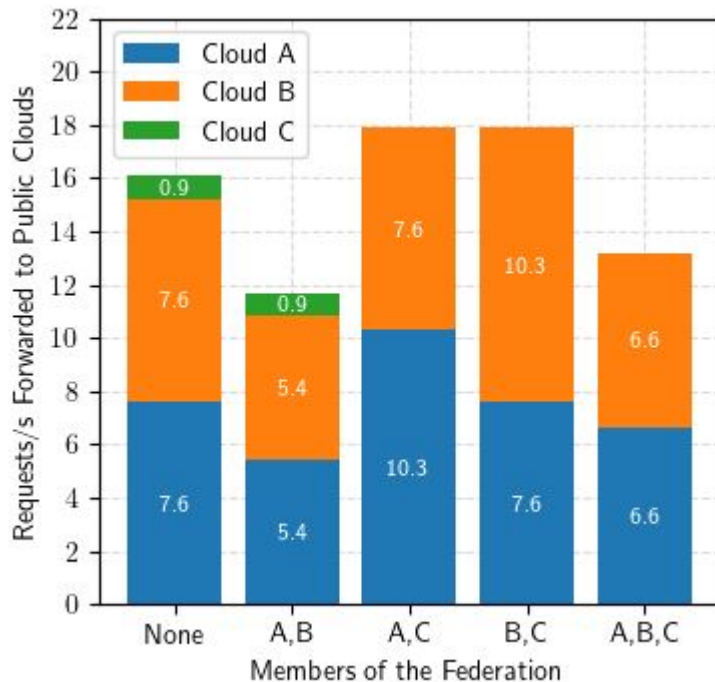
# A Federation of Hybrid Clouds



# Motivating Example

Under the **same arrival rate** (100) and **number of servers** (100), each cloud has individual **latency requirement** (QoS) in unit of mean service time

- Cloud A: 0 (no wait-time)
- Cloud B: 0 (no wait-time)
- Cloud C: 1 (wait upto 1 request)



# Compare to state-of-the-art

Maximizing **performance** by **searching** for the best capacity or workload sharing strategy

- Darzanos 2019, Li 2021: model each cloud as a M/M/1 queue to predict performance.
- Chen 2017, Ray 2021: Invent heuristics to search for nash-stability to satisfy QoS.

Maximizing **Profit** and promote **stability** of a federation

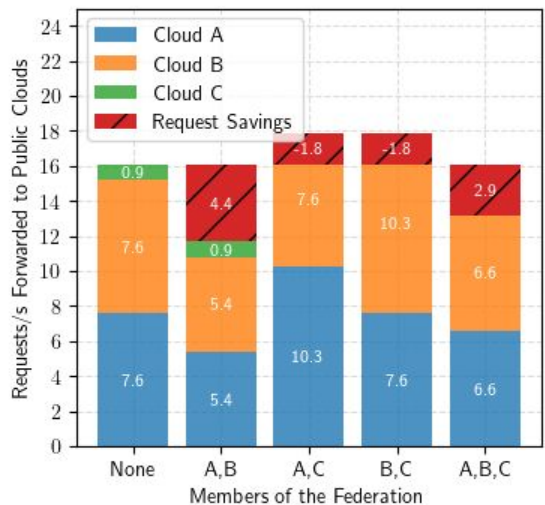
- Lin 2019, Hammoud 2020: Play **repeated game** and **evolutionary game** to let each cloud member adjust sharing strategy.

Maintaining **QoS** (maximum average wait-time)

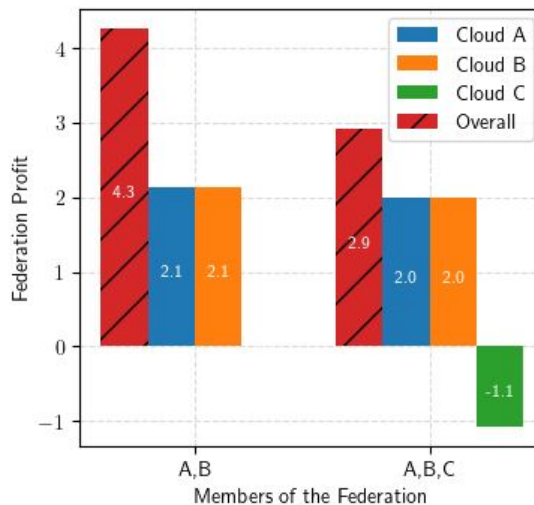
- Ray 2018, Darzanos 2019: **penalize violation** and only consider **homogeneous** QoS.

# Main question

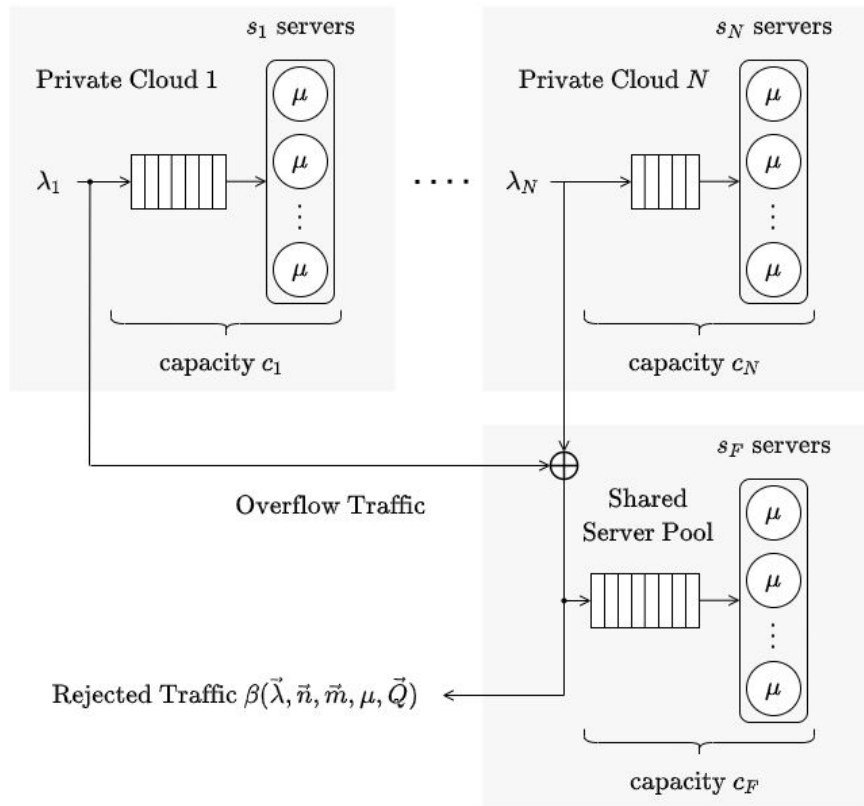
- What is the **optimal sharing strategy** when forming a federation?
- Any reason to **not form** a federation with certain clouds?
- How to **distribute** federation **revenue** according to performance?



Federation Profit Distribution



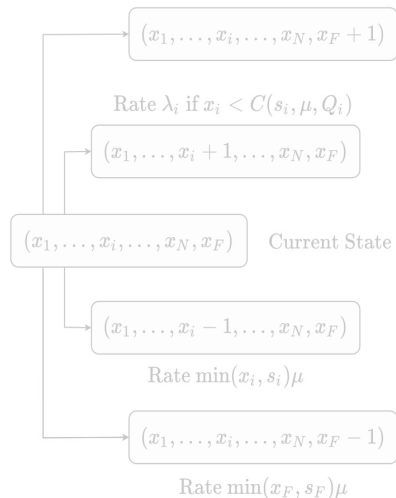
# Performance Prediction



For each cloud  $i$ :

- $\lambda_i$  = arrival rate
- $\mu$  = service rate (globally homogeneous)
- $n_i$  = total number of servers
- $m_i$  = number of servers shared to the federation
- $Q_i$  = latency requirement (QoS) in unit of mean service time

Rate  $\lambda_i$  if  $x_i = C(s_i, \mu, Q_i) \wedge x_F < C(s_F, \mu, Q_i)$



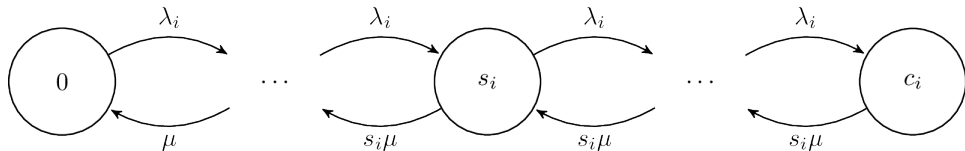


# QoS and Capacity

$$C(s, \mu, Q) = s + q = s + \lfloor \mu Q s \rfloor$$

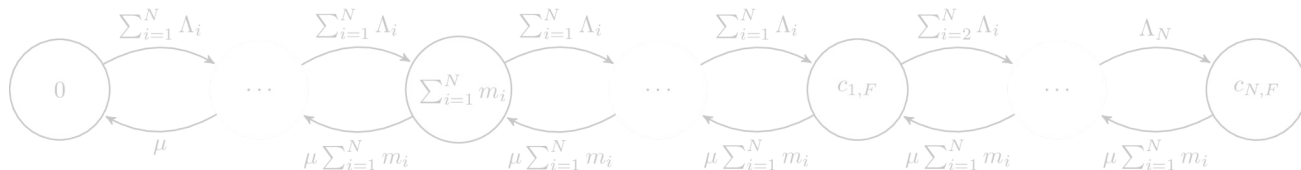
For each cloud  $i$ :

- $s_i = n_i - m_i$  = number of servers available
- $c_i$  = capacity =  $\lfloor (1 + Q_i) \mu s_i \rfloor$

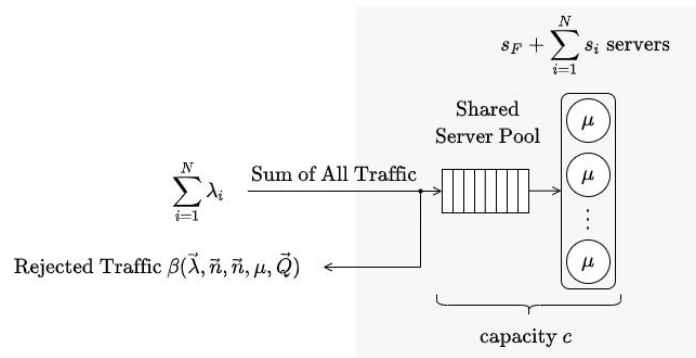
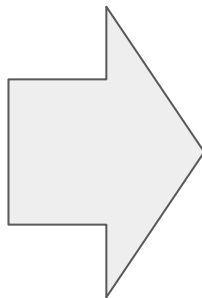
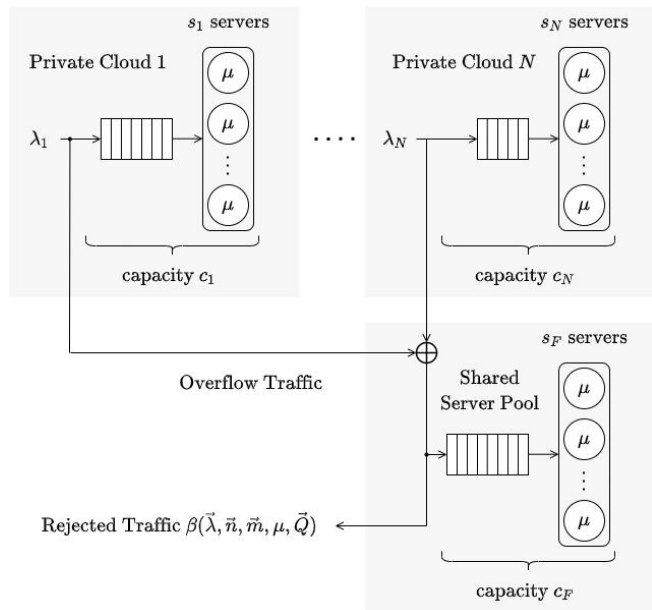


For the federation:

- $\Lambda_i$  = overflow from cloud  $i$
- $c_{i,F}$  = capacity till rejecting  $i$ 's overflow =  $\lfloor (1 + Q_i) \mu \sum_{i=1}^N m_i \rfloor$
- $c_{1,F} \leq c_{2,F} \leq \dots \leq c_{N,F}$
- if  $Q_i = Q \forall i \in \{1, \dots, N\} \rightarrow c_{1,F} = c_{2,F} = \dots = c_{N,F}$



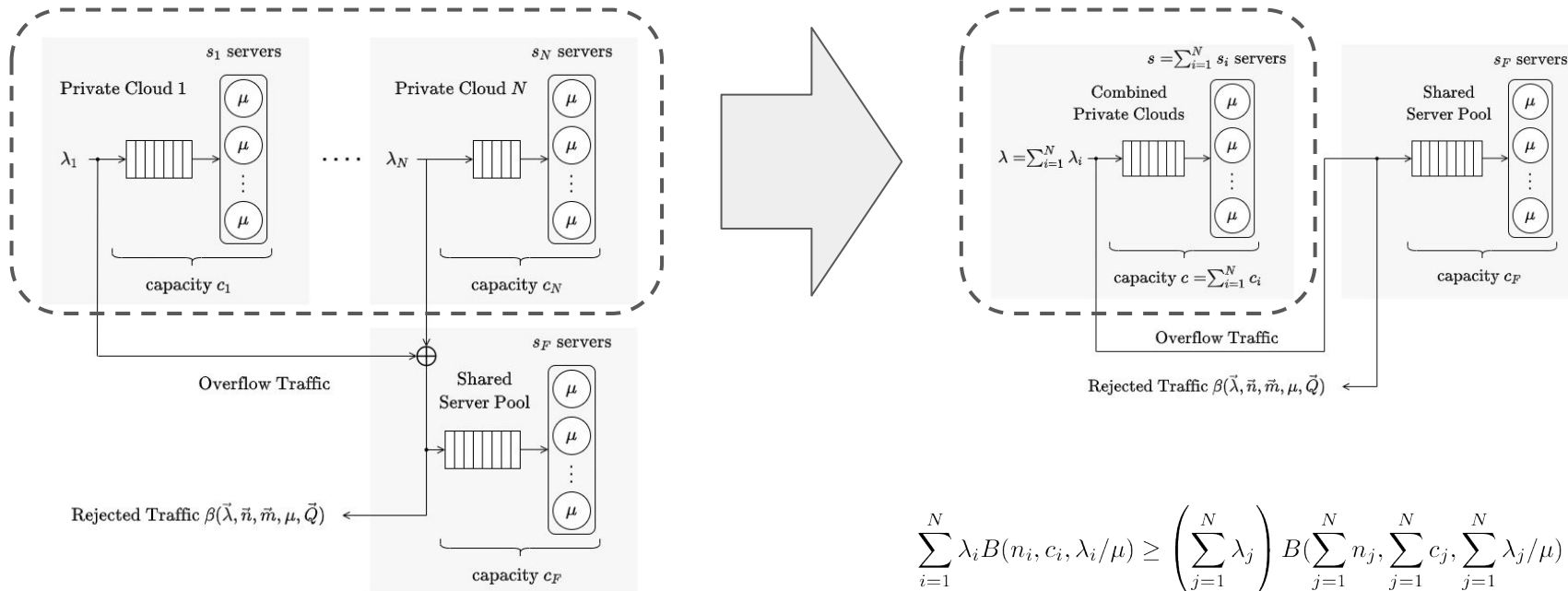
# Optimal Sharing Strategy with Homogeneous QoS



Given  $c_i = \lfloor \mu(1 + Q_i)s_i \rfloor$  and  $c_{i,F} = \lfloor \mu(1 + Q_i) \sum_{i=1}^N m_i \rfloor$ , if  $Q_i = Q$  and  $m_i = n_i, \forall i \in \{1, \dots, N\}$ , then

- $c_{1,F} = c_{2,F} = \dots = c_{N,F}$
- $c = \lfloor \mu(1 + Q)(s_F + \sum_{i=1}^N s_i) \rfloor \geq c_1 + c_2 + \dots + c_N + c_F$

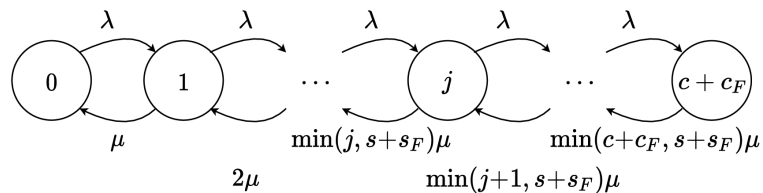
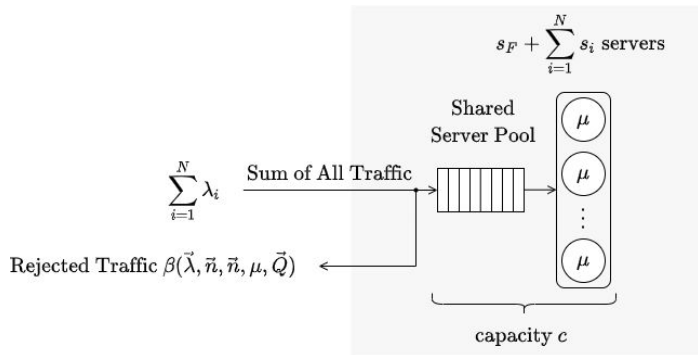
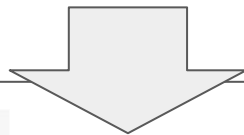
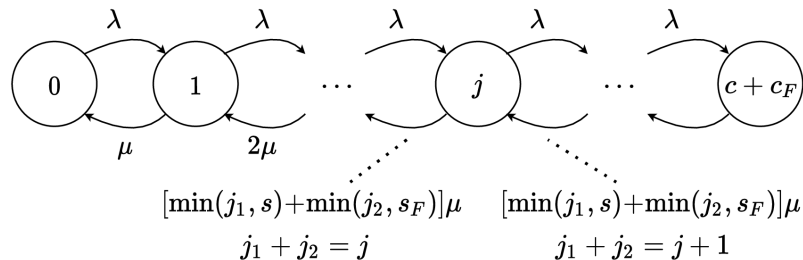
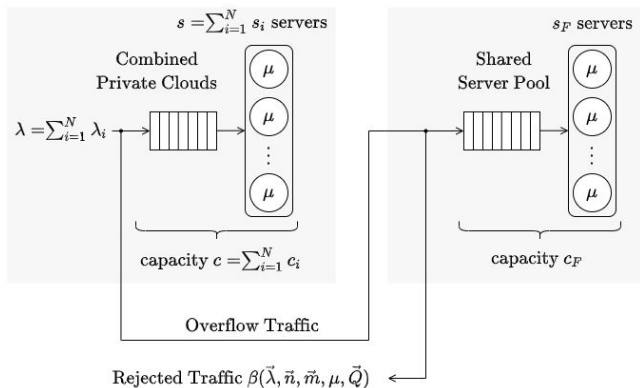
# Minimizing Rejected Traffic



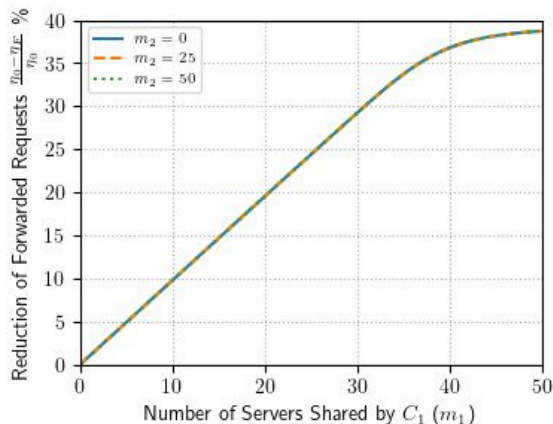
$$\sum_{i=1}^N \lambda_i B(n_i, c_i, \lambda_i / \mu) \geq \left( \sum_{j=1}^N \lambda_j \right) B\left( \sum_{j=1}^N n_j, \sum_{j=1}^N c_j, \sum_{j=1}^N \lambda_j / \mu \right)$$

Smith, David R., and Ward Whitt. "Resource sharing for efficiency in traffic systems." *Bell System Technical Journal* 60.1 (1981): 39-55.

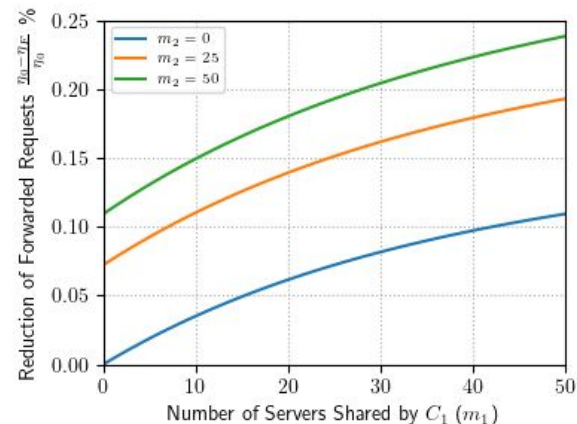
# Minimizing Rejected Traffic



# Sharing Strategy for Homogeneous QoS

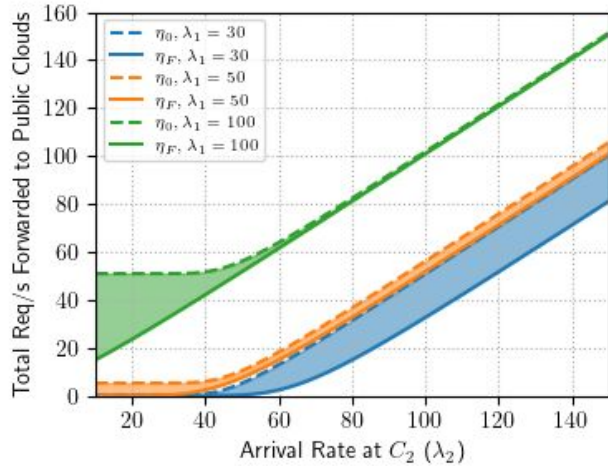


Given an over-provisioned ( $C_1$ ) and an under-provisioned ( $C_2$ ) clouds, sharing more  $C_2$  does not reduce rejected traffic (all of its resources are in use).



With 2 equally overloaded clouds, sharing either can help with the federation.

# Performance (Sharing Everything)

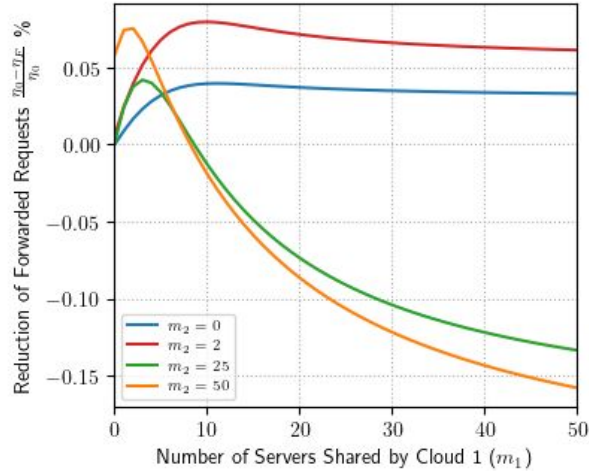


Best to group under-provisioned and over-provisioned clouds together when  $Q1=Q2$ .

| $(\lambda_1, \lambda_2)$ | $\eta_0$ | $\eta_F$ | $\delta_1$ | $\delta_2$ |
|--------------------------|----------|----------|------------|------------|
| (30, 150)                | 100.5    | 81.2     | 27.6%      | 7.1%       |
| (50, 150)                | 105.7    | 101.0    | 5.9%       | 1.8%       |
| (100, 150)               | 151.4    | 150.7    | 0.4%       | 0.3%       |

Table I: Forwarding rate to public clouds before ( $\eta_0$ ) and after ( $\eta_F$ ) joining the federation and relative cost reduction ( $\delta_1, \delta_2$ ) for different arrival rates when  $\vec{n} = \vec{m} = (50, 50)$ ,  $\vec{Q} = (0, 0)$ .

# Heterogeneous QoS (Sharing Everything)

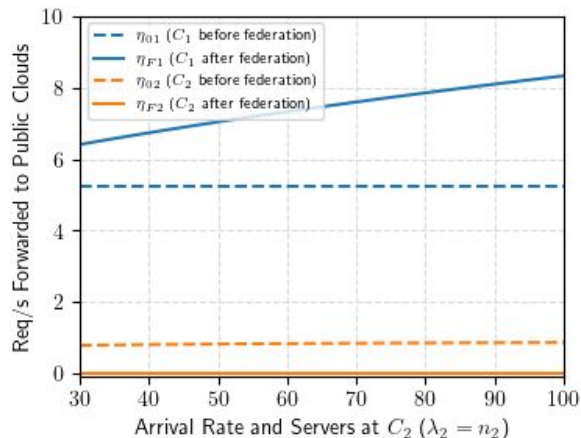


When  $Q_1 = 0$  and  $Q_2 = 1$ , sharing more Cloud 2 servers causes more overflow from 2 and pushes 1's traffic out of federation more than 2 let in.

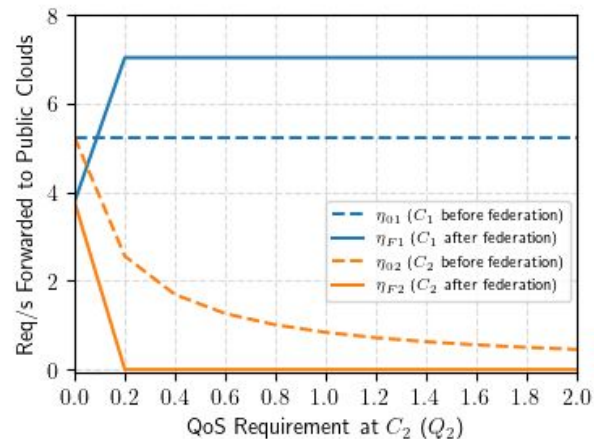
| $(\lambda_1, \lambda_2)$ | $(n_1, n_2)$ | $(Q_1, Q_2)$ | $\delta_1$ | $\delta_2$ |
|--------------------------|--------------|--------------|------------|------------|
| (50, 100)                | (50, 100)    | (0, 10)      | -3.7%      | -2.1%      |
| (50, 100)                | (50, 100)    | (0, 50)      | -3.8%      | -2.2%      |
| (50, 1000)               | (50, 1000)   | (0, 50)      | -14.8%     | -0.8%      |
| (7, 10000)               | (10, 10000)  | (0, 1000)    | -37.4%     | -0.05%     |

Table II: Relative cost reduction with heterogeneous QoS

# Heterogeneous QoS



Given  $Q_1 = 0$  and  $Q_2 = 1$ , increasing Cloud 2's traffic and servers pushes more traffic from 1 to the public cloud.



With  $Q_1 = 0$ , increasing  $Q_2$  quickly allows Cloud 2 to send 0 traffic to the public cloud, pushing traffic from 1 out of the federation but no further.



# Profit Sharing

- Cloud Profit (CP) = revenue (constant) - operating cost (constant) - public cloud cost
- $\Delta CP$  = public cloud cost **before** federation - public cloud cost **after** federation
- Federation Profit =  $R(S) = P\left(\eta_0(S) - \eta_F(S)\right) = \sum_{i=1}^S \Delta CP_i$

Reward policies:

$$1. R_i^{SV}(S) := \sum_{S' \subseteq S \setminus \{i\}} \frac{|S'|! (|S| - |S'| - 1)!}{|S|!} (R(S' \cup \{i\}) - R(S'))$$

Change in the federation profit

$$2. R_i^{SR}(S) := \frac{m_i}{\sum_{j=1}^K m_j} R(S)$$

Shared resources

$$3. R_i^{SIR}(S) := \frac{m_i(1-\rho_i)}{\sum_{j=1}^K m_j(1-\rho_j)} R(S)$$

Shared idle resources

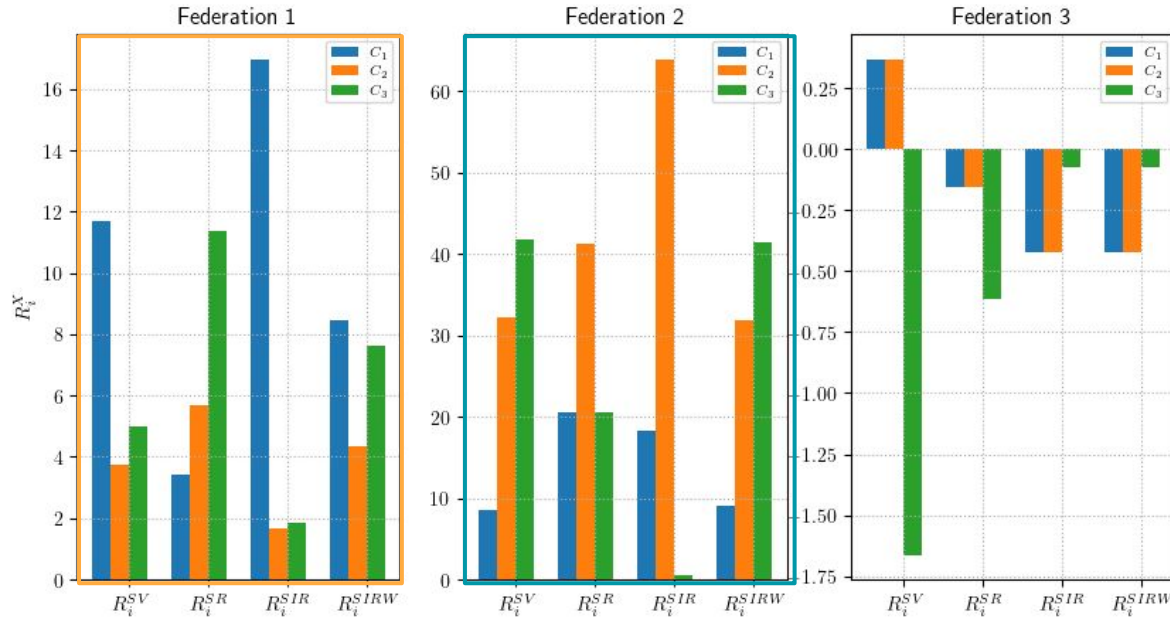
$$4. R_i^{SIRW}(S) := \frac{1}{2} \left( R_i^{SIR}(S) + \frac{\lambda_i - \rho_i n_i \mu}{\sum_{j=1}^K \lambda_j - \rho_j n_j \mu} R(S) \right)$$

Both in shared idle resources and traffic overflow to the federation

# Profit Sharing

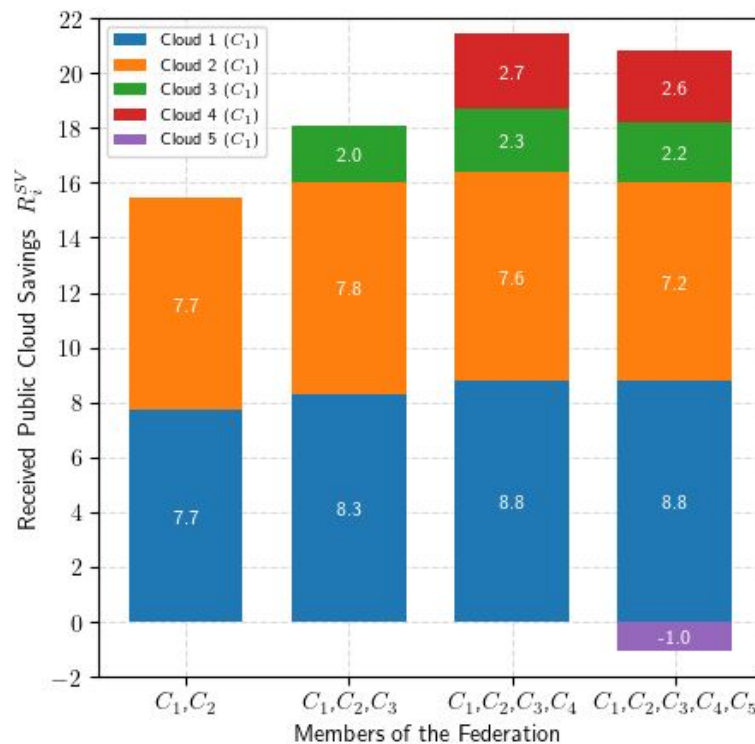
| Cloud | Arrivals  | Servers   | Shared Servers | QoS   |
|-------|-----------|-----------|----------------|-------|
| 1/2/3 | 10/70/140 | 30/50/100 | 30/50/100      | 0/0/0 |
| 1/2/3 | 30/30/150 | 50/100/50 | 50/100/50      | 0/0/0 |
| 1/2/3 | 50/50/200 | 50/50/200 | 50/50/200      | 0/0/1 |

| Contribution          | Federation 1      | Federation 2      | Federation 3      |
|-----------------------|-------------------|-------------------|-------------------|
| Shared Resources      | $C_3 > C_2 > C_1$ | $C_2 > C_1 = C_3$ | $C_3 > C_1 = C_2$ |
| Shared Idle Resources | $C_1 > C_3 > C_2$ | $C_2 > C_1 > C_3$ | $C_1 = C_2 > C_3$ |
| Overflow Traffic      | $C_3 > C_2 > C_1$ | $C_3 > C_1 > C_2$ | $C_1 = C_2 > C_3$ |



# Adding/Discouraging Members

| Cloud | Arrivals | Servers | QoS |
|-------|----------|---------|-----|
| 1     | 10       | 30      | 0   |
| 2     | 50       | 30      | 0   |
| 3     | 30       | 30      | 0   |
| 4     | 50       | 50      | 0   |
| 5     | 50       | 50      | 1   |



# Summary of Contributions

- Evaluate effects of sharing strategies in performance of federations.
- Prove that clouds with the same QoS can maximize federation performance and profit by sharing everything.
- Propose profit sharing mechanism using public cloud saving to compute Shapley Values and illustrate ability to discourage “free riders”.

## Future Work

- Heterogeneous workloads and resources
- Different distributions of workload sizes and interarrival times
- Define more stringent or dynamic queueing capacity to closely respect the exact QoS